

Attention Is All You Need

A simple, visual explanation of the paper that changed AI forever

1. What Problem Does This Paper Solve?

Before 2017, the best language models used **Recurrent Neural Networks (RNNs)** and **LSTMs**. These models read text one word at a time — like reading a book one letter per second. This was painfully slow on long sentences and hard to train because information from early words had to travel through many steps to reach later words.

The authors asked: *What if we throw away the step-by-step processing entirely and just use attention?* The result was the **Transformer** — an architecture that processes all words simultaneously and became the foundation of GPT, BERT, and virtually every modern AI language model.

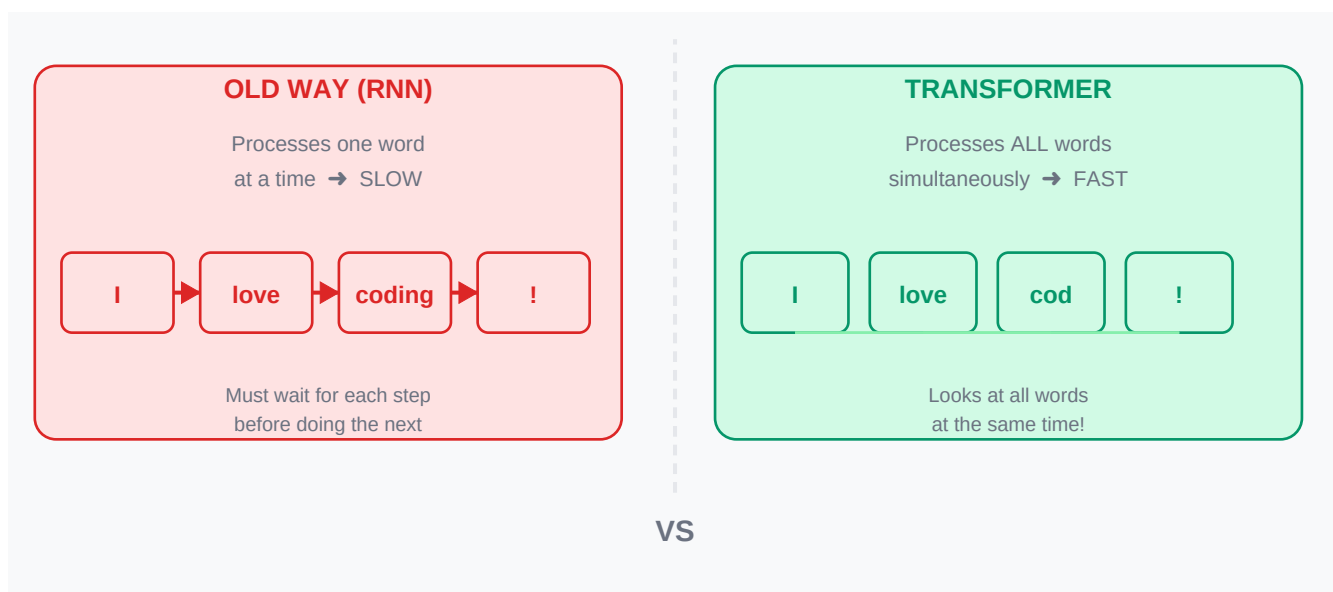


Figure 1: Old RNN approach (sequential) vs. Transformer (parallel processing)

2. The Transformer Architecture

The Transformer follows the classic **Encoder–Decoder** design used in translation. Think of it as two towers: the Encoder reads and understands the input, and the Decoder generates the output word by word.

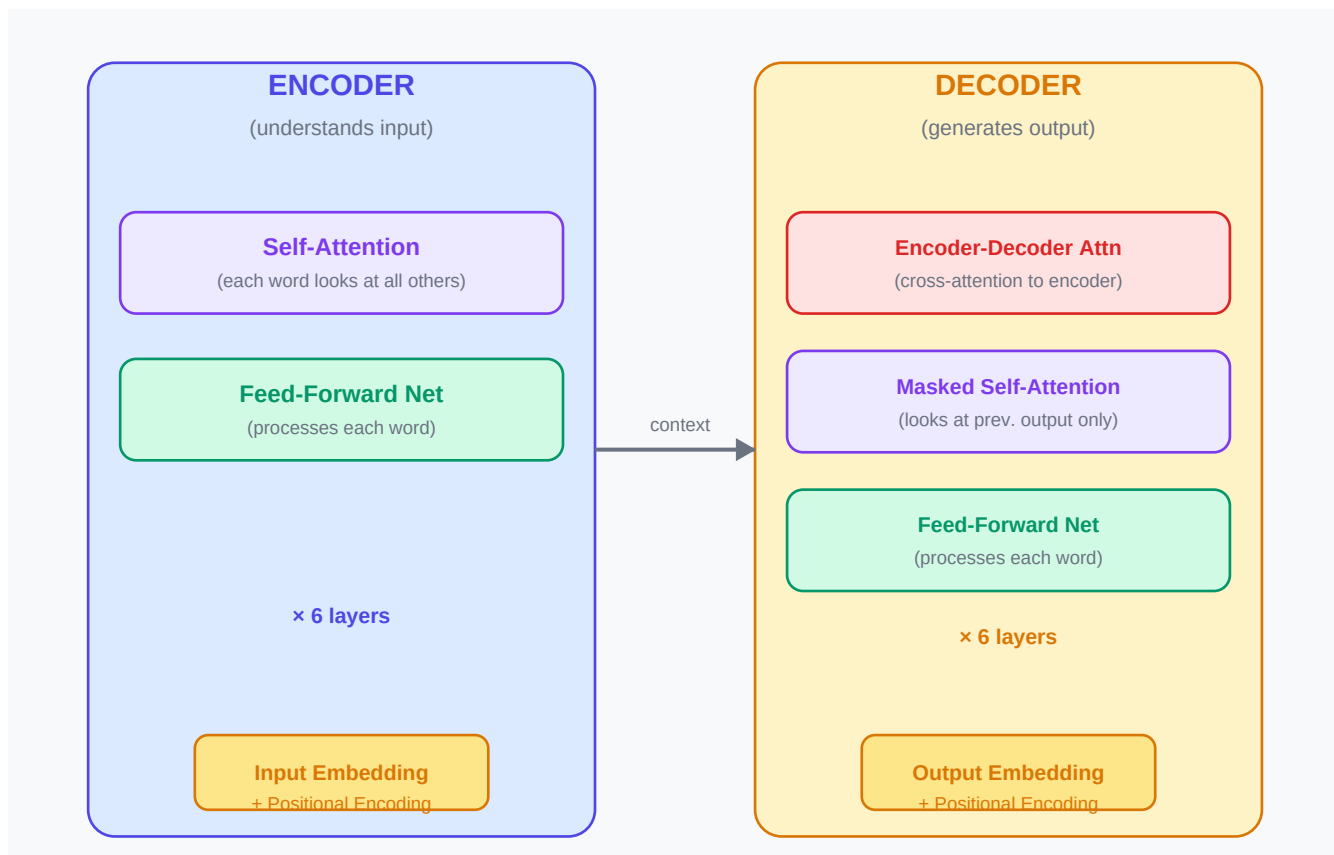


Figure 2: Encoder (left) and Decoder (right) towers, each with 6 identical layers

Encoder

6 identical layers stacked on top of each other. Each layer has two components:

- **Multi-Head Self-Attention** — every word looks at all other words to understand context.
- **Feed-Forward Network** — a simple neural network applied to each word separately.
- **Residual connections + Layer Norm** — think of these as shortcuts that keep gradients healthy during training.

Decoder

Also 6 layers, but with an extra component:

- **Masked Self-Attention** — can only look at previous output words (can't cheat and look ahead).
- **Cross-Attention** — looks at the Encoder's output to connect input and output.
- **Feed-Forward Network** — same as in the encoder.

■ Key insight: all sub-layers use dimension **d_model = 512**, so the output of every layer is the same size and they can be easily stacked.

3. What is Attention? (The Core Idea)

Attention lets a word "look at" other words and decide how relevant each one is. The paper uses a clever analogy from information retrieval:

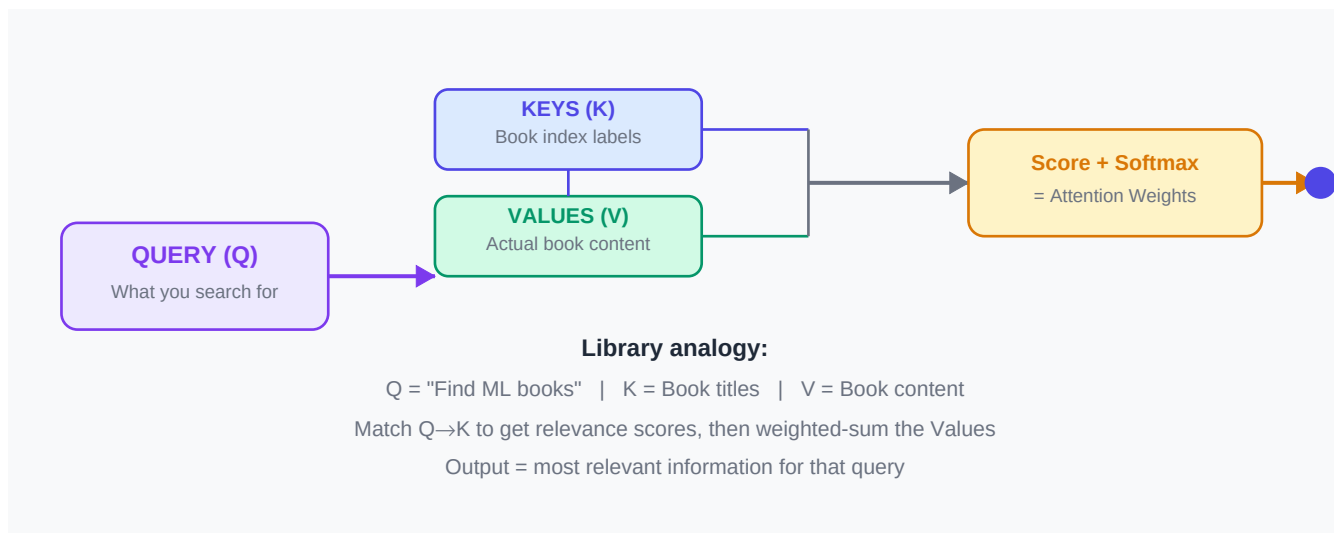


Figure 3: Attention as a library search — Query matches against Keys to produce weights, which are applied to Values to get the output

Scaled Dot-Product Attention

The formula is: $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k}) \cdot \mathbf{V}$

- **Q (Query)** — what this word is looking for
- **K (Key)** — what each word offers as a label/identifier
- **V (Value)** — the actual content each word provides
- **$\sqrt{d_k}$ scaling** — divides by square-root of key dimension to prevent dot products from getting too large (which would push softmax into very small gradients)

4. Multi-Head Attention — Different Perspectives

Instead of running attention once, the Transformer runs it **8 times in parallel** (called "heads"). Each head can learn a different type of relationship between words — one might learn grammar, another might learn word meaning, another coreference, etc.

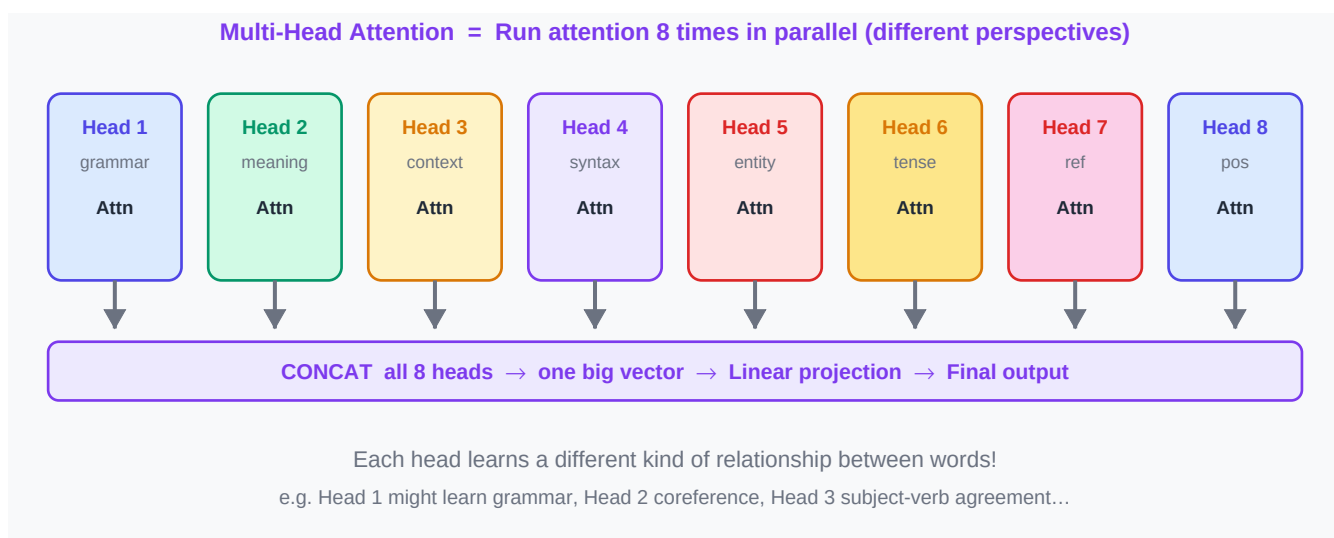


Figure 4: 8 attention heads run simultaneously, each learning different word relationships

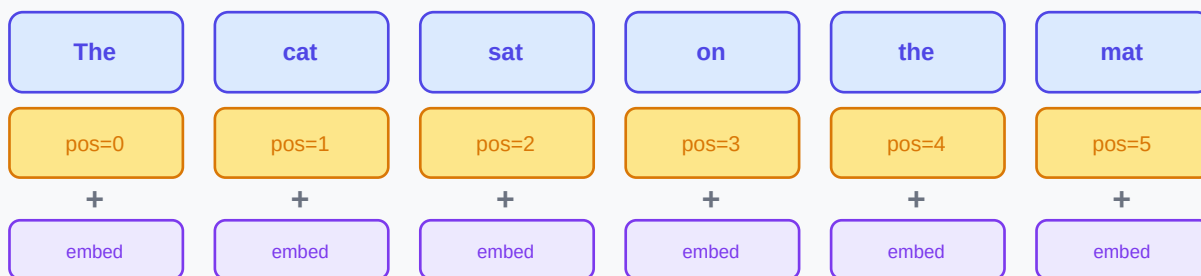
The outputs of all 8 heads are concatenated and projected back down to $d_{\text{model}}=512$. This gives the model much richer understanding than a single attention pass could provide. Dimensions per head: $d_k = d_v = 512/8 = 64$, so total compute stays the same.

5. Positional Encoding — Knowing Word Order

Since the Transformer has no recurrence, it has no built-in sense of order. "The cat sat" and "Sat cat the" would look the same without position info. The solution: add a **positional encoding** to each word's embedding.

HOW IT WORKS:

Each word embedding + a unique position signal → model knows word order



Without positional encoding, the Transformer sees a BAG of words — order is lost!

Positional Encoding adds position info using sine/cosine waves so order is preserved.

Figure 5: Each word embedding is added to a unique positional signal before entering the Transformer

The authors use sine and cosine waves of different frequencies: $PE(pos, 2i) = \sin(pos / 10000^{2i/d})$ and $PE(pos, 2i+1) = \cos(pos / 10000^{2i/d})$. This was chosen because it allows the model to easily learn relative positions (since $PE(pos+k)$ can be expressed as a linear function of $PE(pos)$).

■ Interestingly, learned positional embeddings performed nearly identically — but sinusoids were chosen because they can generalize to longer sequences than seen in training.

6. Why Self-Attention is Better than RNNs

Layer Type	Computation	Parallelizable?	Max Path Length
Self-Attention	$O(n^2 \cdot d)$	Yes ■	$O(1)$ ■
Recurrent (RNN)	$O(n \cdot d^2)$	No ■	$O(n)$ ■
Convolutional	$O(k \cdot n \cdot d^2)$	Yes ■	$O(\log_k(n))$

n = sequence length, d = representation size, k = kernel size. Self-Attention wins on path length ($O(1)$) — information can travel from any word to any other word in a single step, making it much easier to learn long-range dependencies.

The key advantage: in a sentence like *'The animal didn't cross the street because it was too tired'*, a human instantly knows 'it' refers to 'animal'. RNNs must pass this information through every step. Self-Attention connects them directly in one step, no matter how far apart they are.

7. Training Details

Detail	Base Model	Big Model
Encoder/Decoder layers	6	6
d_model	512	1024
Attention heads	8	16
Parameters	65 million	213 million
Training time	12 hours	3.5 days
Hardware	8× NVIDIA P100 GPUs	8× NVIDIA P100 GPUs
Optimizer	Adam ($\beta_1=0.9$)	Adam ($\beta_1=0.9$)
Dropout	0.1	0.3

Learning Rate Schedule:

The learning rate was not fixed — it was increased linearly for the first 4,000 steps (warmup), then decreased proportionally to $1/\sqrt{\text{step}}$. This unusual schedule works better than a fixed learning rate for the Transformer.

8. Results — How Well Did It Work?

The Transformer beat every previous model on two translation benchmarks, while being significantly cheaper to train:

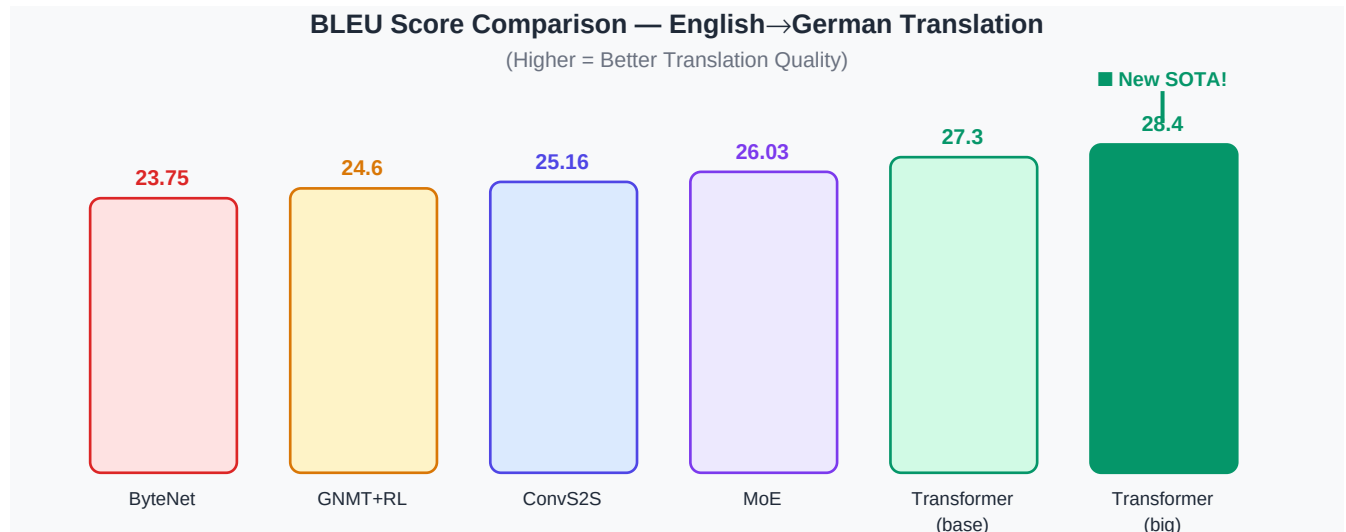


Figure 6: BLEU score comparison on English→German translation. Higher BLEU = closer to human translation quality.

Task	Score	vs. Previous Best	Training Cost
English→German	28.4 BLEU	+ 2.0 BLEU ■	~2.3×10 ¹² FLOPs
English→French	41.0 BLEU	New SOTA ■	< 1/4 of prev. SOTA

9. Why This Paper Changed Everything

This 2017 paper is arguably the most impactful ML paper of the decade. Almost every modern AI you interact with traces back to this architecture:

Model	Company	Based On	What It Does
BERT	Google	Transformer Encoder	Search, NLU
GPT-4	OpenAI	Transformer Decoder	ChatGPT, coding
Claude	Anthropic	Transformer (fine-tuned)	This chat!
T5	Google	Full Transformer	Text tasks
LLaMA	Meta	Transformer Decoder	Open-source AI
Gemini	Google	Transformer (multimodal)	Multi-modal AI

TL;DR — One-Sentence Summary Per Section

Topic	One-line Summary
Problem	RNNs were slow and struggled with long-range word relationships.

Solution	Replace RNNs entirely with an attention mechanism.
Attention	Each word 'votes' on how much to pay attention to every other word.
Multi-Head Attention	Run attention 8 times in parallel so the model learns multiple relationship types.
Positional Encoding	Add sine/cosine position signals so the model knows word order.
Architecture	Stack 6 encoder + 6 decoder layers, each using attention + feed-forward.
Results	Achieved state-of-the-art translation quality at a fraction of the training cost.
Legacy	Every major AI (GPT, BERT, Claude, Gemini) is built on this architecture.

Original paper: Vaswani et al., "Attention Is All You Need", NeurIPS 2017 | Explained for general understanding — some technical simplifications have been made.